StudyForge AI

Beginner Theory and Viva Teaching Guide

Learn the theory first. Then understand your own project code step by step.

For PPT, live demonstration and viva preparation

Start Here

This is not only a project report. It is a teaching book. Read each chapter in this order: first the simple idea, then the real StudyForge example, then the code connection, and finally the questions. If a word is new, do not skip it. The glossary and examples explain it in plain language.

What You Need to Know Before the Viva

- You do not need to be an expert in artificial intelligence.
- You do need to understand how a browser talks to a backend.
- You should know why the project stores data in a database.
- You should be able to explain how an uploaded PDF becomes a chat answer or quiz.
- You should say clearly what is implemented and what is future scope.

Your goal	Simple target
Theory	Explain client, server, API, database, authentication, PDF extraction, RAG and AI.
Code	Know the important files and the responsibility of each.
Demo	Upload a PDF, ask a question, create a quiz and explain the result.
Viva	Answer basic questions in your own words, not memorised jargon.

Contents

1. The Project Story
2. Web Application Basics
3. Frontend Theory
4. Backend Theory
5. Database Theory
6. Security and Authentication
7. PDF Theory
8. RAG Theory
9. Generative AI Theory
10. Study Tool Theory
11. How the Full Project Works
12. Beginner Code Reading Plan
13. PPT Teaching Plan
14. Practice Questions
15. Viva Questions with Answers
16. Glossary
17. Seven Day Study Plan

1 The Project Story

Imagine a BTech student preparing for exams. They have lecture PDFs, notes, small doubts, unfinished revision topics and assignments. Normally these are spread across folders, WhatsApp, notebooks and browser tabs. StudyForge AI brings these jobs into one workspace.

What the Student Can Do

Student action	What StudyForge does	Theory behind it
Sign up	Creates a student record	Authentication and database
Upload PDF	Checks, saves and reads the file	File handling and text extraction
Ask ASHIEY	Finds relevant PDF text and writes an answer	Retrieval plus AI generation
Create flashcards	Makes short revision prompts	Active recall
Create quiz	Makes objective questions and checks answers	Assessment logic
Track revision	Stores topic status	CRUD database operations
Use planner	Stores tasks and dates	User-specific data
Customize avatar	Saves preferences	Frontend state plus JSON data

The One Sentence Project Definition

StudyForge AI is a full-stack study workspace that converts a student's PDF material into interactive learning tools using a React frontend, FastAPI backend, SQLite database, local retrieval and optional generative AI.

Why This is a Full Stack Project

Full stack means the project has both visible user interface and behind-the-scenes server logic. The frontend is the part a student sees in the browser. The backend is the trusted program that checks requests, works with files and database records, and talks to AI services. A project with only a webpage is frontend only; a project with only routes and no interface is backend only. StudyForge has both.

Self Check

Question: What is the main user of StudyForge AI? Answer: A BTech student who wants to learn from their own study PDFs.

Question: What is the input to the core system? Answer: A student account, an uploaded PDF and a question or study action.

Question: What is the output? Answer: Source-based chat, flashcards, quizzes, recall practice, saved progress and organised study records.

2 Web Application Basics

A web application has two major sides. The client is the browser application used by the student. The server is the application that receives requests and performs trusted work. In StudyForge, the browser runs Next.js React on port 3000 and the server runs FastAPI on port 8000.

Client Server Analogy

Think of a restaurant. The customer is the client. The waiter carries an order. The kitchen is the server. The kitchen may use a store room, which is the database. The customer should not enter the kitchen or store room directly. In the same way, the React frontend should not directly open SQLite or use a Gemini secret key.

Term	Easy meaning	StudyForge example
Client	Program used by the user	Next.js React page in browser
Server	Program that receives requests	FastAPI application
API	Rules and URLs used for client-server communication	POST /api/documents/upload
Request	Message sent to server	Create a quiz for this document
Response	Message sent back	Quiz questions or an error message
JSON	Text format for structured data	{ document_id, count }
HTTP method	Type of request	GET reads, POST creates, PUT updates, DELETE removes

API Basics

An API endpoint is a specific URL plus an HTTP method. GET usually reads information. POST creates something or triggers processing. PUT updates existing information. DELETE removes it. For example, GET /api/documents asks for a list of PDFs. POST /api/quizzes/generate asks the backend to create a new quiz.

```
Browser request
POST /api/quizzes/generate
Headers: Authorization: Bearer <signed token>
Body: { "document_id": "abc", "count": 5 }

Backend response
{ "quiz_id": "xyz", "questions": [ ... ] }
```

Question: Why does the frontend not directly read the SQLite database? Answer: Because the database contains private student records. The backend controls access, validates input and keeps secret keys away from the browser.

3 Frontend Theory

The frontend is the visible part of the project. StudyForge uses Next.js, React and TypeScript. Next.js is a framework built around React. React lets developers build a page from small reusable components. TypeScript adds type checking to JavaScript.

What is a Component

A component is a small reusable part of the interface. A page can be made from components just as a house is made from rooms. In StudyForge, AshieyChat is a chat component, Planner is a planner component, Library manages subject materials, and ProfileSetup manages profile data.

React idea	Easy explanation	Project example
Component	Reusable interface piece	AshieyChat, Planner, Library
Props	Data sent into a component	token, user, onSaved
State	Data that can change while page is open	selected PDF, quiz questions, busy
Event	Something the user does	click Create quiz, upload file
useState	React tool for changeable data	const [quiz, setQuiz] = useState
useEffect	React tool for side effects	restore token or load documents
Render	React draws interface from current state	show login or workspace

Understanding page.tsx

frontend/app/page.tsx coordinates the main student experience. It holds current user, token, selected page, document, cards, quiz and notice text. When a user signs in, the page saves a token. When a user selects Quiz, it loads their documents. When a user presses Create quiz, it calls the backend.

```
const [quiz, setQuiz] = useState([])
const [documentId, setDocumentId] = useState("")
const [busy, setBusy] = useState(false)

if (!user) return <LoginScreen />
return <Workspace />
```

The important idea is state changes cause React to render again. When setQuiz receives generated questions, the component automatically updates and shows question buttons. You do not manually edit browser HTML.

CSS and Design

CSS controls colors, spacing, fonts, layout and animation. StudyForge uses several CSS files including atelier.css, workspace.css, dark.css and profile.css. This keeps styles separated by purpose. The warm creative style uses ivory, lavender and ink shades so the interface feels less like a generic dashboard.

Question: What is the difference between React and CSS? Answer: React decides what elements exist and how they respond to state. CSS decides how those elements look and move.

Question: Why use TypeScript? Answer: It can catch mistakes in data shape before the app runs. For example, a Question type says every question should have ID, text and options.

4 Backend Theory

The backend is the trusted side of the application. StudyForge uses FastAPI, a Python web framework. FastAPI maps API URLs to Python functions, validates input and returns JSON.

Routes and Functions

```
@app.get("/health")
def health() -> dict:
    return { "status": "healthy", "project": "StudyForge AI" }
```

The decorator tells FastAPI which URL uses the function. A request to /health runs health. The function returns a Python dictionary and FastAPI sends it as JSON. This endpoint is useful because a launcher can check whether the backend is alive.

Pydantic Validation

A Pydantic model describes expected request data. FastAPI reads incoming JSON, checks it against a model and gives the route a clean Python object. For example, GenerateRequest allows document_id and limits count from 1 to 20. This prevents invalid counts before quiz code starts.

Backend concept	Why it matters
Route	Connects API URL to code.
Pydantic model	Checks request body shape.
Dependency	Runs shared work such as authentication before route.
HTTPException	Returns a clear status code and message.
Database connection	Reads or writes persistent data.
Service/provider function	Calls Gemini or Groq without mixing everything into UI.

Error Handling

Good backend code does not only handle success. It handles missing login, wrong PDF type, too large file, unreadable PDF, provider outage and no valid questions. Status codes make the reason understandable to both frontend and developer.

Question: What is a backend route? Answer: A Python function connected to a URL and HTTP method, such as POST /api/quizzes/generate.

Question: Why validate count from 1 to 20? Answer: It prevents someone from requesting zero, negative or excessively large sets that could waste resources.

5 Database Theory

A database stores information so it remains available after a page refresh or server restart. SQLite is a relational database stored in a local file. A relational database uses tables, rows and columns.

Table, Row and Column

A table is like a spreadsheet category. The users table stores students. One row stores one student. A column stores one property such as email or password_hash. A primary key uniquely identifies a row. A foreign key or linking field connects related records.

Database word	Simple meaning	StudyForge example
Table	Collection of same kind of record	users
Row	One record	one student
Column	One property	email
Primary key	Unique identifier	user ID
Relationship	Connection between records	documents.user_id
CRUD	Create, Read, Update, Delete	signup, list documents, update profile, delete task

Important StudyForge Tables

Table	Purpose
users	Student identity, password hash, profile, subject preferences and avatar.
documents	PDF filename and owner.
chunks	Extracted PDF text plus source page.
conversations	Saved student chat history.
subjects and document_subjects	Organise study PDFs.
student_tasks and revision_topics	Planner and revision tracker.
flashcards	Generated card pairs.
quizzes and quiz_attempts	Question set and score history.

The relationship between users and documents is one-to-many: one user can own many documents. A document has a single owner. The backend checks this relationship before it returns or creates study material.

Question: Why not keep everything only in browser local storage? Answer: Local storage is browser-specific, easy to lose and not suitable for protected shared server logic. SQLite keeps structured data on the backend.

6 Security and Authentication Theory

Authentication answers who are you. Authorization answers what are you allowed to access. In StudyForge, signup and login authenticate a student. owned_document authorizes access to a particular PDF.

Password Hashing

A password should never be stored as readable text. StudyForge uses PBKDF2 HMAC SHA 256 with a random salt and 120000 iterations. A salt is random extra data added before hashing. It means two people with the same password still get different stored values.

Token Flow

18. Student logs in with email and password.
19. Backend checks saved password hash.
20. Backend creates signed token containing user ID and expiry.
21. Frontend saves token in local storage.
22. Frontend sends token in Authorization header.
23. Backend current_user checks signature and expiry before protected action.

The token in this project is a custom signed token using base64 and HMAC SHA 256. It is not a JWT library token. For a college prototype this demonstrates the concept, but production work would use well-tested standard session or JWT tooling.

Question: What is authentication? Answer: Proving which student is making a request.

Question: What is authorization? Answer: Checking whether that authenticated student can access a particular resource.

Question: Why are API keys not stored in frontend? Answer: Browser code can be viewed by users. Secret keys belong in backend environment variables.

7 PDF Theory

A PDF is the student source material. Uploading a PDF is not the same as understanding it. The backend must validate it, save it, extract text, break it into chunks and remember which page each chunk came from.

Upload Validation

Check	Current rule	Why
File type	application/pdf only	Avoid unsupported files.
File size	Up to 20 MB	Avoid uncontrolled resource use.
Page count	1 to 200 pages	Keep processing practical.
Encryption	Rejected if password protected	Cannot reliably extract text.
Owner	User ID saved with document	Protect privacy.

Text Extraction

PyMuPDF opens the uploaded file and reads text from each page. This works well for text-based PDFs. A scanned document is often just an image. It may have little or no text, so local extraction can be weak. That is why OCR is a future improvement.

Chunking Theory

A chunk is a small part of a larger document. StudyForge uses about 180 words per chunk and 30 words overlap. The overlap is important. If a definition ends near the boundary, the next chunk can still include some words from that definition.

```
Page 3 text → chunk 1: words 1 to 180
              chunk 2: words 151 to 330
              overlap: words 151 to 180
```

Question: Why save the source page? Answer: So the app can tell the student where a retrieved explanation came from and improve trust in answers.

8 RAG Theory

RAG means Retrieval Augmented Generation. It has two jobs. Retrieval finds relevant source text. Generation turns that source text into an answer. Without retrieval, a general model may answer from broad training knowledge and ignore the student PDF.

TF IDF in Easy Language

TF means term frequency: how often a word appears in a chunk. IDF means inverse document frequency: a word gets less importance if it appears in every chunk. Therefore, a specific word such as deadlock may get more importance than a common word such as system.

Cosine Similarity in Easy Language

The question and each chunk become number lists called vectors. Cosine similarity asks whether two vectors point in a similar direction. A high score means their important words are similar. StudyForge selects the highest scoring chunks.

Step	What happens
1	Student asks: What is a page fault?

2	Backend loads chunks from selected document.
3	TF IDF converts question and chunks to word-weight vectors.
4	Cosine similarity scores each chunk.
5	Best chunks become source excerpts.
6	Gemini or fallback uses those excerpts for answer.

RAG is not magic. It improves grounding but does not guarantee every answer is correct. The quality depends on readable PDF text, relevant chunks, model behaviour and clear student questions.

Question: What is the difference between retrieval and generation? Answer: Retrieval selects evidence. Generation writes language using evidence.

Question: Why is RAG useful for a study project? Answer: It keeps answers closer to the student notes and allows page references.

9 Generative AI Theory

A large language model, or LLM, predicts useful next text based on input. Gemini is the main document-aware provider in StudyForge. Groq is available as an optional streaming path. The project does not train a new AI model. It uses existing AI services through APIs.

Prompt Inputs

Input	Why it is included
Student question	Tells the model what to answer.
Study mode	Requests simple explanation, hint, code review or viva practice.
Conversation history	Lets follow-up questions make sense.
Retrieved excerpts	Provides relevant source evidence.
PDF when selected	Lets configured provider inspect full material.

Streaming

Streaming means the backend sends answer parts as they arrive. Instead of waiting for a whole long response, the student sees status and growing text. StudyForge sends status, delta and done events using newline-delimited JSON.

Fallback

External AI can be busy, rate-limited or unavailable. StudyForge has a fallback_answer that can display retrieved source text. It also has a local quiz fallback. This is good engineering because the interface explains the situation and preserves useful behaviour.

Question: Did StudyForge train Gemini? Answer: No. It calls an external Gemini API. The project controls prompt, context, validation and display.

Question: Why is a fallback useful? Answer: It keeps a study tool available when external service fails.

10 Study Tool Theory

Flashcards, quizzes and recall games are not just decorations. They support active recall. Active recall means trying to remember before looking at the answer. It is usually more useful for revision than only rereading notes.

How Flashcards Work

The backend tries to generate cards from the selected PDF. It creates a question, short answer, focus label and source page. The frontend lets the student reveal the answer, move next or previous, and shuffle.

How Quiz Work

24. Student selects a PDF and presses Create quiz.
25. Backend checks that PDF belongs to student.
26. Backend requests broad candidate questions from provider.
27. Backend validates question, four options, correct answer, explanation and source page.
28. Duplicate filter keeps distinct concepts.
29. Questions are saved server-side; answers are hidden from initial response.
30. Student submits selected options; backend calculates score and XP.

Why Repetition Filtering Exists

An AI can write the same idea with different words. StudyForge removes stop words, compares meaningful word overlap and also uses sequence similarity. It returns fewer questions rather than inventing repetitive material if a document has too few unique concepts.

Local Quiz Fallback

If the AI provider returns a temporary busy error, make_local_quiz tries to create questions from sentences already extracted from the PDF. The correct answer is a source sentence. Distractors come from other topics. This is more literal, but it is grounded and useful.

Question: Why should a quiz response hide correct answers? Answer: If answers are returned before submission, a user can inspect them in browser data. Server-side answers protect quiz practice integrity.

Question: What does XP mean here? Answer: A simple progress score: each correct answer adds 10 XP and level is calculated from correct answers.

11 How the Full Project Works

Signup to Quiz Example

31. The student opens port 3000, which loads the React frontend.
32. The student signs up. FastAPI hashes the password and inserts a user row in SQLite.
33. The backend sends a signed token. React stores it and sends it on future requests.
34. The student uploads a PDF. FastAPI validates it, saves original file and inserts document and chunk records.
35. The student enters Quiz page. React requests document list using the token.
36. The student chooses a document and clicks Create quiz. React sends document ID and requested count.
37. FastAPI checks ownership, generates or falls back to local questions, stores full quiz and sends safe fields.
38. React renders answer buttons. Student submits choices. FastAPI scores them and returns explanation and source pages.
39. React displays score and progress.

Where to Find Each Part of Code

Question from examiner	File to open	What to point out
How login works?	backend/app/main.py	password_hash, password_matches, create_token, current_user
How PDF upload works?	backend/app/main.py	upload_document, split_text
How RAG works?	backend/app/main.py	retrieve using TF IDF and cosine similarity
How quiz works?	backend/app/main.py	generate_document_items, make_local_quiz, generate_quiz
How interface works?	frontend/app/page.tsx	state, auth, generate, CardsView and QuizView
How chat works?	frontend/app/ashiey-chat.tsx and main.py	fetch/stream plus backend chat routes
How profile/avatar works?	profile setup and avatar creator files	PUT profile/avatar and component state

12 Beginner Code Reading Plan

Do not read every line from top to bottom. Follow one flow at a time. Read a function, write one sentence about its input, processing and output, then test it in the running project.

Day	Read	Learn	Practice
1	README and start-studyforge.ps1	What starts and which ports are used	Run launcher and health route.
2	frontend/app/page.tsx	React state and fetch calls	Trace Create quiz click.
3	main.py models and auth	Pydantic, hash and token	Explain current_user.
4	main.py upload and retrieve	PDF validation, chunking, TF IDF	Draw RAG diagram.
5	main.py chat	Provider, stream and fallback	Explain status, delta, done.
6	main.py quiz and cards	Validation, duplicates, fallback, scoring	Create quiz and inspect result.

7	components and CSS	How code becomes interface	Prepare screenshots and speaking notes.

How to Explain a Function

40. Input: what value does it receive?
41. Validation: what does it check?
42. Processing: which database, PDF or provider operation occurs?
43. Output: what does it return or save?
44. Failure: what happens on error?

Example: generate_quiz receives document ID, count and current user. It checks ownership. It prefers Gemini questions, then local fallback. It saves complete questions in SQLite and returns only safe question fields. It reports 422 if it cannot create a useful distinct quiz.

13 PPT Teaching Plan

Do not make your PPT a copy of code. Slides tell a story; your words explain details. Use screenshots of your own application and one simple architecture diagram.

Slide	Visual	Say this
1 Title	StudyForge name and logo	We built a BTech study workspace.
2 Problem	Student with PDFs and scattered tasks	Revision from notes is manual and fragmented.
3 Solution	One workspace screenshot	StudyForge combines materials, doubts and revision.
4 Features	Feature icons	Upload, chat, cards, quiz, planner, tracker and profile.
5 Architecture	Browser to backend diagram	React calls FastAPI, which works with SQLite, PDFs, retrieval and AI.
6 PDF and RAG	Six-step flow	Text is extracted, chunked, retrieved and used as evidence.
7 Backend	Route and database diagram	Validation, authentication and ownership protect student data.
8 Frontend	Workspace screenshot	React state makes modules update after action.
9 Reliability	Quiz screenshot	Duplicate filtering and fallback avoid repetitive or blocked practice.
10 Testing	Build or test screenshot	We checked backend routes and frontend production build.
11 Limits	Simple table	OCR, semantic search and PostgreSQL are future work.
12 Closing	Thank you	Explain value and invite questions.

Simple Opening Script

Good morning. Our project is StudyForge AI, a study workspace for BTech students. Students often have notes, PDFs, doubts and revision tasks in different places. We built one system where a student uploads their own PDF and then uses it for source-based chat, flashcards, quizzes, recall practice, revision tracking and planning.

Simple Closing Script

StudyForge AI is not only a chatbot. It combines a React frontend, FastAPI backend, SQLite storage, PDF extraction, local retrieval and optional generative AI. The system also protects student-owned documents and provides fallback study tools when an external AI service is unavailable.

14 Practice Questions Before Viva

Answer these without looking at the answers. Mark the ones you cannot explain. Then return to the matching theory chapter.

45. What is the difference between frontend and backend?
46. What is an API endpoint?

47. Why does StudyForge use two ports?
48. What does JSON mean in this project?
49. What does React state do?
50. What does useEffect do?
51. What is FastAPI used for?
52. What is Pydantic validation?
53. What is SQLite?
54. What is a table, row and primary key?
55. What is authentication?
56. What is authorization?
57. Why do we hash passwords?
58. What is a token?
59. Why should AI keys remain on backend?
60. Why validate PDF type and size?
61. What is text extraction?
62. What is a chunk?
63. Why use chunk overlap?
64. What is RAG?
65. What is TF IDF?
66. What is cosine similarity?
67. What is a source page?
68. What is an LLM?
69. What does streaming mean?
70. What is a fallback?
71. Why filter duplicate questions?
72. Why hide quiz answers?
73. What is active recall?
74. What does XP measure?
75. What is a 422 status?
76. What is a 503 status?
77. What is a test?
78. What would you improve for production?

15 Viva Questions with Beginner Answers

Question: Explain the whole project. Answer: StudyForge AI is a full-stack BTech study workspace. React shows the interface. FastAPI processes requests. SQLite stores student records. PDFs are extracted and split into chunks. TF IDF retrieval finds relevant chunks. Gemini can create answers and study material, while local fallback keeps core study behaviour available.

Question: What is full stack? Answer: It means both frontend and backend are present. StudyForge has a React frontend plus FastAPI backend and SQLite database.

Question: Why use FastAPI? Answer: It makes Python API routes easy, validates request data with Pydantic, supports dependency injection and gives automatic API docs.

Question: Why use React? Answer: React builds screens from reusable components and automatically updates the interface when state changes.

Question: What is Next.js? Answer: A framework around React that gives project structure, build tools and web application features.

Question: Why use SQLite? Answer: It is simple, local and suitable for a BTech prototype. A larger deployment can move to PostgreSQL.

Question: What is RAG? Answer: It retrieves relevant document text before generation so answers stay closer to student material.

Question: How does your retrieval work? Answer: PDF chunks and the question are converted using TF IDF. Cosine similarity ranks chunks, and the top relevant chunks are used as sources.

Question: What is TF IDF? Answer: It gives higher weight to words that are important to a specific text and lower weight to common words.

Question: What is cosine similarity? Answer: It compares vector direction; a higher result means the question and chunk use similar important words.

Question: Why chunk a PDF? Answer: Small chunks are easier to search. Overlap prevents losing an idea at a boundary.

Question: How does upload work? Answer: Backend checks token, file type, size, page count and encryption. PyMuPDF extracts text, then chunks are saved with page number.

Question: How is a document protected? Answer: It stores owner user ID. Before using it, backend checks current user and document ownership.

Question: How are passwords handled? Answer: The backend uses PBKDF2 HMAC SHA 256 with salt and iterations. It stores only a hash, not raw password.

Question: What is the difference between authentication and authorization? Answer: Authentication proves the student identity. Authorization checks whether that student may use a particular document.

Question: Is your token JWT? Answer: No. The current project uses a custom HMAC signed token with expiry. Standard session or JWT tooling is future production work.

Question: What happens when Gemini is busy? Answer: Chat can show retrieved source text and quiz can use a local source-grounded fallback generated from PDF sentences.

Question: How do you avoid repeated quiz questions? Answer: The backend generates extra candidates, compares meaningful word overlap and sequence similarity, then keeps different concepts.

Question: Why do you keep quiz answers server-side? Answer: The frontend receives only questions and options until submission. This prevents answers being visible before student attempts quiz.

Question: Why use streaming? Answer: It gives status and answer text gradually, so the student gets feedback while a long answer is being generated.

Question: Is your avatar true 3D WebGL? Answer: No. It is a 3D-styled React and CSS design. This gives a lightweight custom look without a heavy rendering engine.

Question: What are project limitations? Answer: Scanned PDFs may have weak text, TF IDF is lexical rather than semantic, providers can be unavailable and SQLite is local scale.

Question: What are future improvements? Answer: OCR for scans, embeddings and vector search, PostgreSQL, standard sessions, rate limiting, caching and usability evaluation.

16 Glossary

Word	Easy meaning
API	A set of URLs and rules used by frontend and backend.
Backend	Trusted server program.
Client	Browser application used by student.
Component	Reusable React UI part.
Cosine similarity	Number showing vector similarity.
CRUD	Create, Read, Update, Delete.
Database	Persistent organised storage.
Endpoint	One API URL and method.
FastAPI	Python framework used for backend.
Frontend	Visible browser interface.
Hash	One-way transformed password value.
HTTP	Web communication protocol.
JSON	Structured text sent in request and response.
LLM	Large language model.
Pydantic	FastAPI data validation model.
RAG	Retrieval Augmented Generation.
SQLite	Local relational database.
State	Data that can change in a React component.
TF IDF	Word weighting for retrieval.
Token	Signed proof of logged-in user.

17 Seven Day Study Plan

Day	Main focus	Output
1	Project story and web basics	Speak one-minute explanation.
2	React and frontend	Explain state, components and page.tsx.
3	FastAPI and routes	Explain request, response, Pydantic and errors.

4	Database and authentication	Explain SQLite, tables, hash and token.
5	PDF, chunking and RAG	Draw upload and retrieval flow.
6	AI, quiz and fallback	Explain provider, streaming and distinct questions.
7	PPT, demo and viva	Rehearse slides and 23 model answers.

Final Beginner Reminder

Do not try to sound advanced by using words you cannot explain. Start with the student journey. Then say what the frontend does, what the backend does, what the database stores and how PDF retrieval helps AI answer. If asked about an unimplemented advanced feature, say it is future scope. Honest understanding is better than exaggerated claims.